Agentes de IA con Pydantic AI

De cero a producción con menos magia y más ingeniería.

De qué trata

La charla recorre cómo construir un agente de IA con Python sin tratarlo como magia. Un agente se escribe como cualquier otro servicio: con tipos, dependencias explícitas, pruebas y límites. El material usa un caso sencillo, el triage de peticiones de soporte de una aplicación, con datos sintéticos.

Qué es un agente

Un agente es un modelo que elige qué herramienta usar, recibe el resultado y decide de nuevo, hasta tener una respuesta. Ese ciclo, el *agent loop*, es la base de todo lo demás.

El stack

Cuatro piezas y una capa que las observa.

- **FastAPI** recibe la petición y la valida con Pydantic.
- **Pydantic AI** define el agente: modelo, herramientas y salida con tipos. Lo desarrolla el equipo de Pydantic.
- **Pydantic** valida los datos en cada frontera: la petición, los argumentos de las herramientas y la salida del agente.
- **PostgreSQL** guarda los datos de la aplicación y, si hace falta, el historial de mensajes o vectores con pgvector.
- **Logfire**, del mismo equipo de Pydantic, traza FastAPI, Pydantic, Pydantic AI y las consultas a la base.

La idea que une el stack es que los tipos hacen de contrato entre las piezas.

Un agente tipado

La salida es un modelo Pydantic. El agente declara su `output_type` y el tipo de sus dependencias. La herramienta recibe esas dependencias por `RunContext`.

```
 class Triage(BaseModel):
    category: Literal["facturacion", "acceso", "error", "otro"]
    urgent: bool
    needs_review: bool = Field(description="True si falta información")
    summary: str

agent = Agent(
    deps_type=Deps,
    output_type=Triage,
    instructions="Clasifica el ticket. Si falta información, marca needs_review.",
)

@agent.tool
async def customer_plan(ctx: RunContext[Deps], customer_id: int) -> str:
```

```
    plan = await ctx.deps.db.customer_plan(customer_id)
    return plan or "desconocido"
```

El harness

El modelo aporta capacidad; el harness la convierte en un sistema controlable. Un harness es el sistema que rodea al modelo y organiza el contexto, el *agent loop*, las herramientas, la validación, los límites y el registro de lo ocurrido. Pydantic AI aporta piezas del harness, no toda la arquitectura de producción.

Agentes de dominio

No diseñes un agente para todo. Diseña uno excelente para algo. Un agente de dominio hace bien un trabajo concreto, con contexto y herramientas acotados, y su resultado se puede evaluar. Si falta información, pasa el caso a revisión en lugar de inventar una decisión.

FastAPI y el agente

El servicio expone el agente en un endpoint. FastAPI valida la petición, el agente corre dentro del servicio y devuelve una salida validada. El plazo total se impone desde fuera con `asyncio.timeout`, y un semáforo acota la concurrencia.

Producción

Escalar es operar con control bajo carga.

1. **Pruebas y evaluaciones.** Con `TestModel y agent.override` se prueba sin red.
2. **Límites y plazos.** Un plazo total y un máximo de ejecuciones simultáneas.
3. **Observabilidad.** Trazas de cada ejecución, herramienta y validación.
4. **Servicio asíncrono.** Concurrencia acotada.

Para llevarse

- Los tipos son el contrato entre las piezas.
- El harness hace fiable al modelo.
- Un agente, un trabajo.
- Validar la forma de una salida no prueba que sea verdadera.